



---

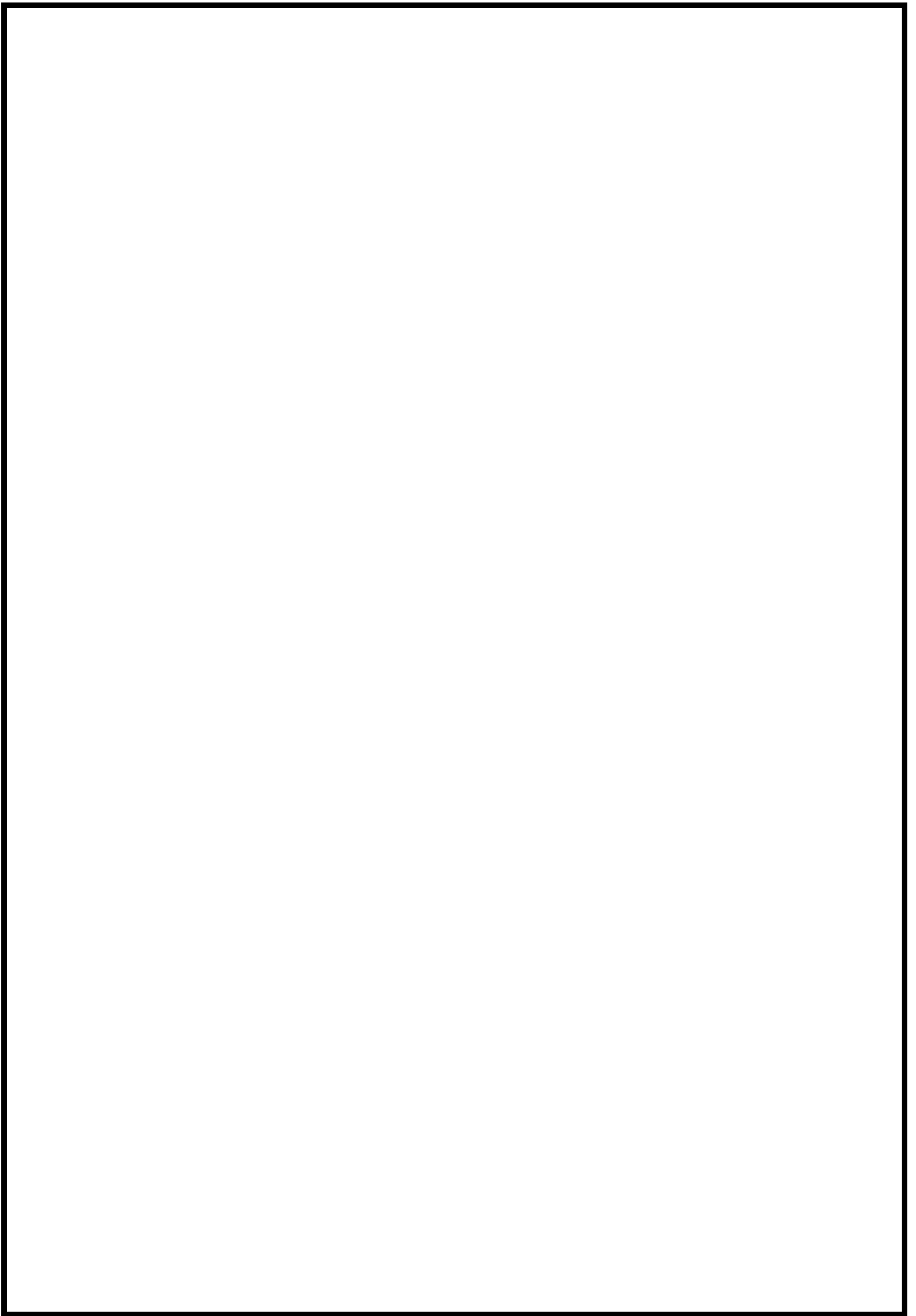
# DAA

---

Project report



|                     |                    |
|---------------------|--------------------|
| <b>Submitted To</b> | <b>Mam Samra</b>   |
| <b>Submitted By</b> | <b>Abdul Mohiz</b> |
| <b>CMIS ID</b>      | <b>534658</b>      |
| <b>Course</b>       | <b>BESE 30-B</b>   |
| <b>Date</b>         | <b>05 May 2026</b> |



# 1. Introduction

Dynamic pricing is a modern and widely adopted strategy used in industries such as e-commerce, ride-sharing services, airlines, and hospitality to maximize revenue and improve resource utilization. The core idea behind this approach is to continuously adjust product or service prices based on changing market conditions rather than keeping them fixed. These conditions may include customer demand, time factors, availability of resources, and even competitor pricing.

In the context of online food ordering platforms, dynamic pricing can play an especially important role. Restaurants often face fluctuating demand throughout the day—for example, peak hours during lunch and dinner versus low demand during late hours. At the same time, ingredient availability and preparation costs may also vary, directly affecting profitability. By applying a dynamic pricing model, food delivery systems can intelligently modify menu prices in real time to reflect these changing conditions.

This project focuses on designing and implementing an intelligent pricing algorithm that adjusts food item prices based on key factors such as customer demand patterns, time of day, and ingredient availability. The goal is not only to maximize revenue for the platform or restaurant but also to maintain a balanced level of customer satisfaction by ensuring fair and reasonable pricing.

From a Design and Analysis of Algorithms (DAA) perspective, this system incorporates efficient algorithmic techniques to process data and make optimal pricing decisions. It combines greedy strategies for quick adjustments, optimization methods to improve profit margins, and predictive modeling concepts to estimate future demand trends. By integrating these approaches, the system aims to provide a practical and scalable solution for real-world dynamic pricing challenges in the food industry.

## 2. Problem Statement

Design a dynamic pricing algorithm that adjusts menu prices in real time based on demand, time of day, and ingredient availability. The goal is to maximize revenue while ensuring efficient resource use and maintaining customer satisfaction.

## 3. Objective

The dynamic pricing system is designed using the following key techniques:

### **Greedy Algorithm:**

This technique is used for making fast and immediate pricing decisions. It adjusts prices based on current conditions such as high demand, low demand, or limited stock. Since it focuses on local (instant) decisions rather than long-term planning, it is very suitable for real-time systems like food pricing.

### **Optimization Techniques:**

These techniques are used to ensure that the overall system generates maximum possible profit. Instead of just adjusting prices individually, optimization considers all items together and finds a pricing strategy that increases total revenue while respecting constraints like minimum/maximum price limits, demand levels, and stock availability.

### **Machine Learning Models:**

Machine learning is used to predict future customer demand based on historical data such as past orders, time of day, and food popularity. This helps the system make intelligent pricing decisions in advance rather than reacting blindly.

### **Feedback Loop System:**

The feedback loop continuously improves the system by learning from real customer behavior. After prices are updated, actual sales data is compared with predicted demand. This error is then used to adjust the model and improve future predictions and pricing decisions.

## **4. Methodology**

### **4.1 Algorithm Description and Implementation**

The algorithm operates in four sequential stages: Data Collection and Preprocessing, Demand Prediction, Greedy Price Adjustment, and Feedback Loop Optimization. Each stage processes and refines the output of the previous one, forming a continuous and iterative pricing pipeline. This pipeline ensures that pricing decisions are data-driven, adaptive, and continuously improved based on real-time system feedback.

## Algorithm:

```
INPUT: D = dataset of food items, orders, stock levels
OUTPUT: Updated price list P for all food items

1: Load dataset D with n food items and m historical orders
2: Preprocess D: normalize prices, encode time slots, encode categories
3: Train ML demand prediction model on historical data
4: for each time slot t in T do
5:     Predict demand  $d_i$  for each food item i
6:     Compute ingredient availability score  $a_i$  for item i
7:     GREEDY PRICE ADJUSTMENT:
8:     for each food item i do
9:         if  $d_i > \text{demand\_threshold}$  then
10:            Increase price:  $P[i] = P[i] * (1 + \alpha)$ 
11:        else if  $a_i < \text{stock\_threshold}$  then
12:            Increase price:  $P[i] = P[i] * (1 + \beta)$ 
13:        else
14:            Decrease price:  $P[i] = P[i] * (1 - \gamma)$ 
15:        end if
16:        Apply constraint:  $P_{\min} \leq P[i] \leq P_{\max}$ 
17:    end for
18:    OPTIMIZATION: Maximize Revenue =  $\text{SUM}(P[i] * d_i)$ 
19:    Apply FEEDBACK LOOP: Update model with observed demand
20: end for
21: return P
```

### Step 1: Data Collection & Preprocessing

- Collect historical order data, including timestamps, food items, quantity ordered, and price paid.
- Divide time into meaningful slots such as Breakfast, Lunch, Dinner, and Late Night for better analysis.
- Normalize numerical values such as order quantity, price, and ingredient stock levels to ensure consistency.
- Encode categorical features such as food item category, payment method, and customer type into numerical form for processing.

**Time Complexity:**  $O(n)$

**Space Complexity:**  $O(n)$

### Step 2: Demand Prediction using ML Model

A supervised machine learning model, such as Random Forest or Linear Regression, is trained on historical order data to predict the demand for each menu item across different time slots.

- The model is trained using labeled features mapped to demand values.
- It predicts the demand score  $d_i$  for each item  $i$  at the current time slot  $t$ .

- The greedy pricing strategy uses this predicted demand: items with higher demand scores lead to price increases, while lower demand results in price reductions.

**Time Complexity:**  $O(m \log m)$  for training,  $O(n)$  for prediction

**Linear Regression is used to:**

- Predict **continuous demand values** for each food item
- Capture **basic relationships** between:
  - Time of day (breakfast, lunch, dinner)
  - Price of item
  - Historical order frequency
- Provide a **fast and lightweight prediction model**

**Pseudo Code**

```
INPUT: Dataset D with features X (time, price, category, stock) and target demand Y
OUTPUT: Trained weight vector W

1: Initialize weights W randomly
2: Add bias term to feature matrix X
3: for each iteration do
4:     Predict demand:  $\hat{Y} = W * X$ 
5:     Compute error:  $E = Y - \hat{Y}$ 
6:     Update weights:
7:          $W = W + \alpha * (X^T * E)$ 
8: end for
9: return W
```

**Random Forest is used to:**

- Predict **demand more accurately using multiple decision trees**
- Handle **non-linear relationships** such as:
  - Sudden demand spikes during peak hours
  - Complex customer behavior patterns
- Reduce **overfitting** by averaging multiple models

## Pseudo Code

```
INPUT: Dataset D, number of trees N
OUTPUT: Final demand prediction model

1: for i = 1 to N do
2:   Create bootstrap sample  $D_i$  from D
3:   Build decision tree  $T_i$  using  $D_i$ 
4:   At each node:
5:     Randomly select subset of features
6:     Choose best split using impurity (Gini/Entropy)
7:     Split dataset into child nodes
8: end for

9: To predict demand for input  $x$ :
10:  for each tree  $T_i$  do
11:    compute prediction  $P_i = T_i(x)$ 
12:  end for
13:  Final prediction  $d_i = \text{average}(P_i \text{ for all trees})$ 
14: return  $d_i$ 
```

### Step 3: Greedy Price Adjustment

The Greedy Algorithm performs local, real-time price adjustments at each time step to improve revenue without requiring full global optimization. Pricing decisions are based on the following conditions:

- **High Demand ( $d_i > \text{threshold}$ ):** Price is increased by a factor  $\alpha$  (e.g., 10–20%).
- **Low Stock ( $a_i < \text{threshold}$ ):** Price is increased by a factor  $\beta$  to control and discourage excessive orders.
- **Low Demand with Normal Stock:** Price is decreased by a factor  $\gamma$  to attract more customers.
- **Constraints:** All prices are kept within valid bounds:  $P_{min} \leq P[i] \leq P_{max}$ .

**Time Complexity:**  $O(n)$

**Space Complexity:**  $O(n)$

### Step 4: Optimization & Revenue Maximization

After greedy adjustments, an optimization step evaluates different pricing configurations to maximize total expected revenue while respecting demand and price constraints.

➤ **Objective Function:**

Maximize  $R = \sum P[i] \times d_i$  for all items  $i$

➤ **Constraints:**

$$P_{min} \leq P[i] \leq P_{max}$$

$$d_i \geq 0 \text{ (non-negative demand)}$$

The system selects the price vector that yields the highest estimated revenue.

**Time Complexity:**  $O(n)$

**Space Complexity:**  $O(n)$

### **Step 5: Feedback Loop System**

After each pricing cycle, the system records actual customer behavior and updates the model to improve future predictions, forming a continuous learning loop.

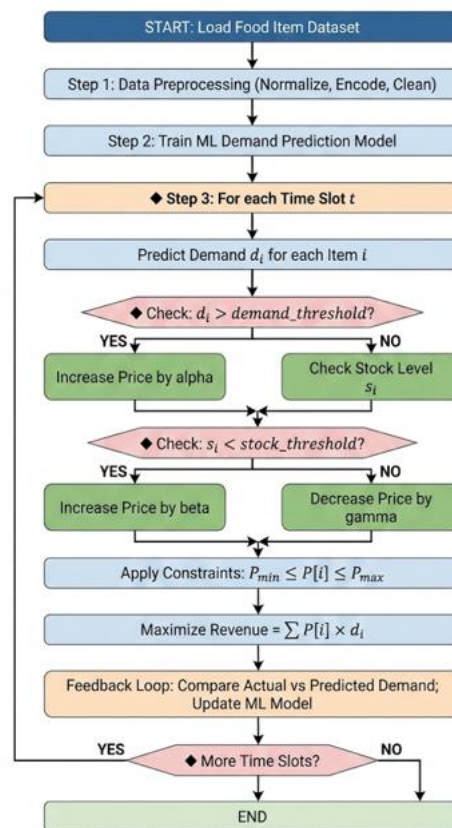
- Record actual orders placed at updated prices
- Compute prediction error:  
**error = |predicted – actual|**
- Incrementally retrain or fine-tune the ML model using new data
- Update pricing parameters ( $\alpha$ ,  $\beta$ ,  $\gamma$ ) based on performance and accuracy

**Time Complexity:**  $O(m)$

**Space Complexity:**  $O(m)$

## 5. System Flowchart

The following flowchart illustrates the end-to-end workflow of the Dynamic Pricing System:



## 6. Mathematical Model

Let:

$P_i$  = price of item  $i$

$d_i$  = predicted demand

$a_i$  = availability factor

**Revenue Function:**

$$R = \sum (P_i \times d_i)$$

**Constraints:**

$$P_{min} \leq P_i \leq P_{max}$$

$$a_i \geq \text{threshold}$$

## 7. Big-O Notation

| Step                 | Algorithm             | Time Complexity | Space Complexity |
|----------------------|-----------------------|-----------------|------------------|
| Data Preprocessing   | Cleaning & Encoding   | $O(n)$          | $O(n)$           |
| Demand Prediction    | ML Model Training     | $O(m \log m)$   | $O(m \times f)$  |
| Demand Prediction    | ML Prediction         | $O(n)$          | $O(n)$           |
| Greedy Adjustment    | Price Update per Item | $O(n)$          | $O(n)$           |
| Revenue Optimization | Objective Function    | $O(n)$          | $O(n)$           |
| Feedback Loop        | Model Update          | $O(m)$          | $O(m)$           |

## 8. Conclusion

The proposed dynamic pricing system integrates multiple algorithmic techniques to achieve real-time responsiveness and revenue optimization. A Greedy Algorithm enables fast, locally optimal price adjustments for each time slot, while a Machine Learning model predicts customer demand with high accuracy. An Optimization layer then determines pricing strategies that maximize overall revenue across constraints such as demand and stock availability. Additionally, a Feedback Loop continuously updates the system using new data, improving prediction accuracy and pricing decisions over time. Overall, the system is scalable, adaptable, and well-suited for deployment in real-time food ordering platforms.

## 9. Future Enhancements

- Implement Deep Learning models such as LSTMs to capture sequential demand patterns across long time horizons and improve temporal forecasting accuracy.
- Apply Reinforcement Learning to enable autonomous discovery of optimal pricing policies through continuous interaction with the environment.
- Integrate competitor pricing data using web scraping techniques to support market-aware and competitive pricing adjustments.
- Deploy the system in a real-time production environment with low-latency API integration into Point-of-Sale systems for immediate price updates.
- Enhance the feedback loop by incorporating A/B testing methodologies to empirically evaluate and compare different pricing strategies for performance improvement.